なぜ WebAssembly (+ Rust) に注目するのか、なぜ実用できていないのか

2022/01/13 TSC 定例

小原 一哉

前書きとアウトライン

この資料では私がなぜ WebAssembly (と Rust) に注目しているのかと注目しているのになぜ実用できていないのかについて説明します。

前半で WebAssembly と Rust の軽い紹介と注目したいと思える特徴について、後半で実用に躊躇しているポイントについて考えを述べます。

私が WebAssembly について一番頑張って調べたのが 2017 年に Qiita に記事を投稿した ときなので古い情報が含まれているかもしれません。

WebAssembly と Rust の紹介

WebAssembly とは

https://webassembly.org/ より

> WebAssembly（略して Wasm）は、スタックベースの仮想マシン用のバイナリ命令フォーマットです。Wasm は、プログラミング言語のポータブルコンパイルターゲットとして設計されており、クライアントおよびサーバーアプリケーションの Web での展開を可能にします。(訳注: 機械翻訳)

一言で言うなら「新しい VM」。初学者にわかりやすい表現は「JavaScript の代わりに配るバイナリ」「Java アプレットのリブート」。

Java バイトコード、LLVM IR と同様、直接書くようなものではない (はず)

WebAssembly の特徴 (1/2)

https://webassembly.org/ より

1. 効率的で高速
2. 安全
3. ソースを開いてデバッグ可能
4. オープンウェブプラットフォームの一部

WebAssembly の特徴 (2/2)

効率的で高速？

そもそものニーズ。テキストのソースコード (JavaScript) を配るよりバイナリ形式の方がサイズとロード時間が小さくなるはず。一般的なハードウェア機能を利用してネイティブ速度で実行できるように目指している。

安全？

ブラウザのネイティブ拡張を捨てざるを得なかった歴史を踏まえて設計されているため、デフォルトで隔離されている。

参考: 出来ることは計算だけ？「WebAssembly」は一体なにが新しいのか〜エンジニアが語る技術愛 #03〜 | ミクシル

WebAssembly の利用例

- Google Meet (参考: なぜ Google Meet の背景ぼかしが最強なのか（一般公開版）)
- Google Earth (参考: (WebAssembly のプロダクション利用をまとめたサイト))
- Figma (参考: (WebAssembly のプロダクション利用をまとめたサイト))
- Photoshop (参考: Photoshop's journey to the web)
- sql.js
- FFMPEG.WASM
- こえのブログ (参考: アメブロ 2019: こえのブログでの PWA | CyberAgent Developers Blog)
- Amazon IVS (最も使われている Wasm ライブラリ) (参考: WebAssembly | 2021 | The Web Almanac by HTTP Archive)
- Unity WebGL (参考: WebAssembly が WebGL ビルドのスタンダードに！ | Unity Blog)

(全部リンク先からの伝聞。実際に調べてないので現在はどうなっているか不明)

Rust とは

https://www.rust-lang.org/ja より

> 効率的で信頼できるソフトウェアを誰もがつくれる言語

新しめのプログラミング言語。パフォーマンスが意識された設計で、ランタイムやガーベッジコレクターがなく高速。型システムと所有権モデルによりメモリ安全性とスレッド安全性が保証される。エコシステム、周辺ツール群も意識されてサポートされているため色々揃ってる。

Wasm を出力できる言語の中でも相性がよい (後述)。

C, C++ の後継というわけではないがたぶんユースケースが最も近いのはそっちあたり。JavaScript, Go が中心の私にとっては新しい概念が多く習得が非常に難しく感じる。

なぜ Wasm と Rust に注目するのか

なぜ Wasm に注目するのか？ (1/2)

- 増加する JS サイズ: 2018 → 2021 で 19 % 近くの増加 (Web Almanac 2021 より)
- もっと高機能に、もっと速く、というニーズ
  - Google のオフィス向けツール, Slack, Miro, GitHub Codespaces など、以前ならデスクトップアプリしか考えられなかったものが Web で提供されるのが当たり前に
  - UX がビジネスインパクトを持つという当たり前の事実 & Google の検索ランキングスコアに速さが含まれるように

(「検索する」という行動様式が変わらない限りおそらく) Web の高機能化の流れは止まらない & UX は常に重要なので高機能化に伴う高速化の需要は高まっていくはず。

なぜ Wasm に注目するのか？ (2/2)

- Web 以外の資産の利用: FFMPEG.WASM など
- Web 以外の利用シーン、新しい VM としての可能性
  - 「安全に任意コードを実行できる環境」としてのポテンシャルの高さ、実績 (ブラウザ上)、使いやすさ (中間表現)
  - Fastly Compute@Edge, Proxy-Wasm
  - WASI
- 言語の壁の突破、大統一言語: Java が見た夢をもう一度

もちろん Web での利用が中心になるはずだけど、Web 以外の利用も考えられる。私はとくに CDN エッジロケーションで動かすスタイルが流行ると思っていた (過去形)

(Wasm を考えるとき) なぜ Rust に注目するのか？

WebAssembly を出力できる手段は多くあるが、Rust はとくに相性が良いように思える。

- Rust 公式でのビルドサポート (Tier 2)、整備されたツール群
- ランタイム (ガーベッジコレクター含む) を持つ言語は、ランタイムそのもののサイズが Wasm の「小さいサイズ」というメリットが薄れさせてしまう
- 抽象度が高く書きやすい文法やライブラリ

なぜ Wasm と Rust を実用できていないのか

なぜ Wasm を実用できていないのか？ (1/2)

- (私がやってきた仕事の中では) 「速さ」の需要がゼロ
- (仮に「速さ」が要求されたとして) Wasm なしのチューニングの余地があり余っている
- Wasm にしただけでは速くならない、というのが通説
  - JavaScript がそもそも異常に速い
  - みんなが使えば Wasm も最適化されていき速くなる期待はできる
- 実績がないから速くできるかわからない、わからないからコストをかけられない、コストをかけられないから実績が作れない、というジレンマ

なぜ Wasm を実用できていないのか？ (2/2)

- Wasm が効果的なユースケースが特殊
  - Wasm だけだと純粋な計算機: DOM は操作できない、通信できない、デバイスは見えないなど
  - Wasm が外部とやりとりするために JavaScript を経由する必要があるが、オーバーヘッドがある: 回数が多いと逆に遅くなる
  - アプリケーションのうち、計算量が多く画面描画や通信などの頻度が入出力が少ない一部を処理するのに向いている: そういった部分がなければ不要
    - Rust and WebAssembly の以前のチュートリアルはライフゲームだった

私が作ってきたアプリケーションで (たぶん) 重さの原因のほとんどが DOM 操作、UI ライブラリ周り = Wasm で解決しない領域。
Google のように高機能なアプリケーションを作るか、ゲームを作るか、動画編集ソフトを作るか…… AI 関連は向いてそう。

なぜ Rust を実用できていないのか？

- **難しい**: 組織的に習得するビジョンが見えない、挑戦する勇気が持てない
  - 元から触れる人は (狙って採用しない限りは) まずいない & 触れるようになるまで時間がかかる & 人の入れ替わりがある
  - 私はこれまでに「なぜかビルドできない」で完全に詰まることが何度かあった
  - 所有権、ジェネリクス、トレイトあたりが最低限わかればアプリケーションは書けるかも。これらだけでも私は苦労した
  - 私自身はまだアプリケーションを書ける気がしていない

Wasm との組み合わせにおいては気になるところはとくにない

楽観的に見ればなんだかんだで「なぜか動く」で使えるかも……

まとめ

- 私の仕事で Wasm が必要なシーンがない
- Rust は難しそう